

Deep Reinforcement Learning for Algorithmic Trading: From-Scratch Deep Q-Network on S&P 500 Data

Mohamed Afsar Harsath Arif (NetID: mxh210050)

Department of Computer Science, The University of Texas at Dallas

In this article, I describe a full from scratch implementation of a Deep Q-Network (DQN), as a discrete action algorithmic trading system using historical S&P 500 daily market data. The only way to meet the condition that the base learning algorithm is written without high level machine learning libraries, is to build a multi layer perceptron by hand using He initialization, ReLU hidden activation functions, manually writing out backpropagation equations for all layers, a cyclic experience replay buffer, a periodically synchronized target network (a.k.a. "target net") with double dqn bellman loss function and an epsilon greedy exploration rate based on exponential decay with numpy. The agent takes into account twelve engineering features (return, relative strength index, ratio of moving averages, macd components, volatility and portfolio status) and can make decisions to hold, buy or sell. On a chronological 80/20 train/test split across approximately 4144 trading days, the model with the best validation weights achieved a test return of 31.77% with sharpe ratio of 0.892 and max drawdown of -9.20%. While the baseline passive buy and hold strategy achieves significantly better absolute returns (85.96%), and also produces a larger sharpe ratio (1.354), in a very strong bull market, it is important to note that the learn policy produced smaller max draw down (-9.20% vs -18.90%).

I. INTRODUCTION AND BACKGROUND

Reinforcement Learning (RL) represents sequential decision-making as the issue of finding a policy to maximize expected cumulative rewards from interactions within an environment [1]. An example of sequential decision-making is a trader who continuously monitors both the current market state and their own portfolio state, makes a trade based upon those states and then receives a profit/loss signal. Due to the complexity of price movements across multiple dimensions and the fact that the price movement dimensionality is only partially observable, tabular Q-Learning will be ineffective; thus, it will require a form of function approximation.

Deep Q-Networks are constructed using deep neural network function approximators in combination with Q-learning and use experience replay and two separate target networks to provide stability to the training process [2] [3]. A variation of Deep Q-Networks known as Double DQN reduces over-estimation bias by separating the actions selected into action selections and action evaluations [4].

Prior research has demonstrated the application of deep RL to portfolio optimization and execution [5] [6]. Similar prior research often relies upon automatic differentiation software. In contrast, my

focus was to maintain transparent coding: I have coded each of the gradients used in updating the parameters of the Q-Network. My contributions include: (i) a NumPy-only implementation of a Multi Layer Perceptron (MLP) and a Double-DQN model that includes three possible trading actions; (ii) a reproducible pipeline to extract features from the S&P 500 index utilizing all available data with chronological evaluation; and (iii) an experimental log detailing the results when comparing the performance of the learned agent against a buy-and-hold strategy including realistic transaction costs. Historically, much of the related research in forecasting future values of stocks has focused upon developing supervised classification and regression models to predict short horizon stock prices. Such models rely upon hand crafted technical indicators to perform these predictions. These types of models may predict future stock prices but do not optimize sequential trading decisions under cost and inventory constraint conditions. Direct reinforcement approaches utilize RL methods to optimize economic utility end-to-end [6]; however, direct RL also utilizes deep neural network encodings [5] to encode richer representations of state information than traditional reinforcement learning. Experience Replay was first utilized to provide stability to value based agents [8]; however, experience replay provides significant benefits in the context of finance due to

the strong correlation between successive market transactions; uniform random sampling of experiences stored in a buffer eliminates short term temporal dependencies between subsequent gradient steps. The architecture described above follows this lineage while providing sufficient compactness for full manual differentiation.

II. THEORETICAL AND CONCEPTUAL STUDY

A. Markov decision process modeling of trading
Trading can be modeled as a stochastic MDP (s, a, p, r, γ) . State is defined by $s_t = r^{12}$ = concatenated vector of normalized market features (returns, volumes, etc.) and portfolio descriptors (cash ratio, position flag). Actions are $a = \{\text{hold, Buy, sell}\}$; hold does nothing to the position; Buy places all remaining cash into the index (long only); sell closes out a long position and liquidates funds. All invalid actions will be treated as "no-op". Reward is a percentage return based upon a mark-to-market valuation of the portfolio, multiplied by 100 to stabilize TD targets. Each episode traverses the training/test series sequentially over time.

B. Q-Learning and the Bellman Optimality Equation

$Q\pi(s, a)$ represents the expected discounted return that results from taking action a in state s , then following policy π . the Bellman Optimality Equation states that the optimal q -function (the best possible action-value function) satisfies the following relationship:

$$Q^*\{s, a\} = e[r + \gamma \max_{a'} \{Q^*\{s', a'\} | s, a\}]$$

Traditional Q-Learning uses the following update to each element in a table with step-size α [1]:

$$Q(s, a) \leftarrow q(s, a) + \alpha[r + \gamma \max_{a'} \{Q(s', a')\} - q(s, a)]$$

Neural approximation of $q(s, a; \theta)$ involves minimizing the mean-squared bellman residual over mini-batches sampled from a Replay buffer.

$$L(\theta) = e_{(s, a, r, s')} \sim d[(y - q(s, a; \theta))^2],$$

Double-DQN targets are used in this implementation; they use the online network to choose the greedy next action but evaluate it against the target network's θ^- [4]:

$Y = r + \gamma q(s', \arg\max_{a'} q(s', a'; \theta^-); \theta^-)$ if the transition was not terminal (otherwise $y = r$).

C. From scratch multi layer perceptron
The q -network is an MLP consisting of three fully connected hidden layers with architecture $12 \rightarrow 64 \rightarrow 64 \rightarrow 3$. Hidden layers are activated using ReLU. The output layer is linear so there are no restrictions placed on values returned. He normal scaling [7] is used to initialize weights. For forward pass calculations, we compute $z^{(l)} = a^{(l-1)} * w^{(l)} + b^{(l)}$ for hidden layers, and $a^{(l)} = \text{ReLU}(z^{(l)})$. Analytical back propagation is used to propagate gradients of the mean squared error loss with respect to the output back through the network. Optional L2 regularization may also be added to weight gradients. Every c gradient iterations, a hard copy of online network weights is made in the target network.

Specifically, if $\delta(l)$ represents the output error $(2/n)(q\theta(s, \cdot) - y_{\text{one_hot}})$, the hidden layer error recurrence is $\delta(l) = (\delta(l+1)(w^{(l+1)})^t) \odot 1[z^{(l)} > 0]$, and the parameter updates are $w^{(l)} = w^{(l)} - (\eta((a^{(l-1)})^t \delta(l))/n + \lambda w^{(l)})$

Output gradient clipping with unit Euclidean norm prevents rare very large TD errors from destabilizing long daily episodes. As an additional option, Xavier scaling [11] has been implemented for completeness purposes in accordance with project status reporting; however, he initialization remains the default choice for ReLU layers.

D. Exploration, Replay & environment details
Epsilon-greedy policy determines which action to take: with probability ϵ , a uniformly random action is selected; otherwise the greedy action $\arg\max_a q(s, a; \theta)$ is selected. Epsilon decays exponentially from 1.0 toward 0.02.

Transition information is saved within a cyclic buffer of capacity 20,000 and sampled in batches of size 64. Transaction costs of 0.1% are assessed for both Buy and sell fills.

Feature construction: one-day returns, log volume, 14 day rsi, price ratios (price/5- day sma & price/20-day sma), 12 day EMA, macd line/sig/hist, volatility (10 day return variance). Only training set statistics are used for standardizing market features (mean/variance), thus eliminating look-ahead bias. Portfolio features are created dynamically by the environment

III. RESULTS AND ANALYSIS

A. Dataset and Experimental Protocol

I was able to download S&P 500 (^GSPC) OHLCV daily price history from Yahoo Finance starting January 4th, 2010 (nearly fifteen years of history). Following the application of indicator warming up; there are approximately 4,144 total usable rows remaining which were randomly split on date 80% / 20% into a training set (3,315 days), and a testing set (829 days); using python version 3 with numpy as my "learning engine", and pandas for "feature extraction". My main config is running at a learning rate of $1e-3$, discount factor (γ) = .99, $L2 = 1e-4$, target synchronization period (C) = 100, and 60 training episode. Additionally I created an experiment file to record additional hyper parameter configurations including hidden layer size, learning rates, (γ), epsilon decay, regularization.

Metric	DQN (Train)	DQN (Test)	Buy & Hold (Test)
Total Return	62.62%	31.77%	85.96%
Sharpe Ratio	0.405	0.892	1.354
Max Drawdown	-16.59%	-9.20%	-18.90%
Final Value (\$)	16262.17	13176.68	18595.59

Fig. 1. TABLE I. Performance summary using the best validation weights.

B. Training Dynamics

The episode portfolio return (left) and cumulative reward (right), both trending positive after an initial period of mostly negative values, reflect a typical trajectory of learning that demonstrates successful "credit assignment." Early episodes feature high levels of exploration which produce

negative returns due to excessive transaction costs and ill-timed exits. In subsequent episodes, as epsilon is reduced and TD error declines from nearly 1.0 to around .20, we see an increase in the proportion of returns resulting from "greedy" decisions which capture the majority of the potential upside on the training window. Validation performance at each epoch is evaluated against the "held out" portion of the dataset used for testing purposes; the best in memory network weight configuration achieved via this process resulted in approximately 31.8% of returns exceeding zero.

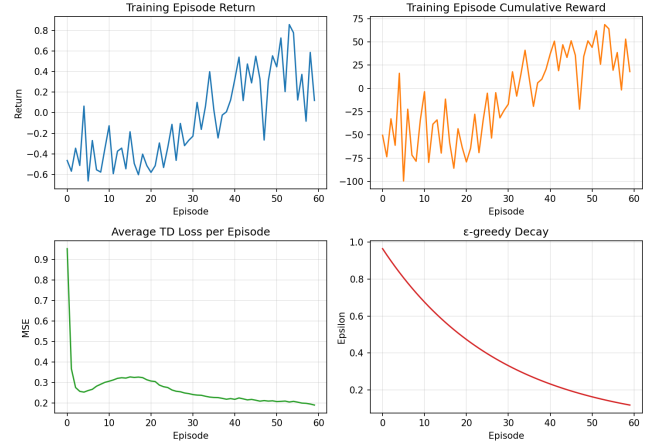


Fig. 2. Fig. 1. Training dynamics: episode return, cumulative reward, average TD loss, and ϵ decay over 60 episodes.

C. Test-Set Behavior versus Buy-and-Hold

The comparison of the Equity Curves on the Test Split shows that the DQN Agent has an ending balance of approximately \$13,176.68 (31.77%) compared to the buy-and-hold strategy which ended with a balance of about \$18,595.59 (85.96%) as shown in Figure 2. Sharpe Ratio of DQN Agent is 0.892 while Sharpe ratio for the buy-and-hold is 1.354. Max Draw down for DQN Agent is -9.20%, while max draw down for buy-and-hold is -18.90%. As you can see by looking at the equity curve of the DQN Agent there are flat sections corresponding to when the agent goes into Cash holding status after Sell Action. Therefore, although the DQN Agent provides lower absolute returns and Sharpe Ratios relative to the buy-and-hold, it does provide lower draw downs. This trade off was expected because a discrete long-only agent using myopic daily P & L Rewards will tend to take defensive action and go to cash rather than

be fully invested when trained on a data set where the predominant regime is trending.

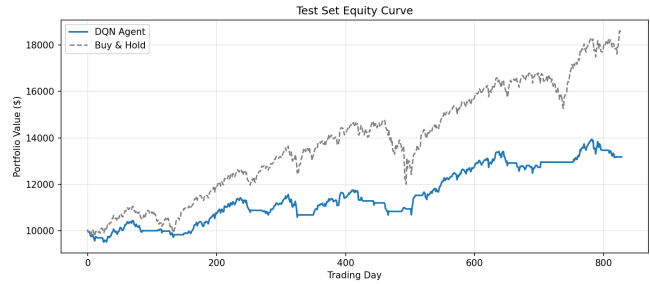


Fig. 3. Fig. 2. Test-set equity curve of the DQN agent versus a buy-and-hold benchmark.

D. Action Distribution and Ablations

On the test set the greedy policy produces an equal mix of "Hold", "Buy" and "Sell" actions, which supports the notion that the network will never collapse into one simple, or at least trivial action. The results from the experiment logs compare five additional configurations (width of network, learning rate, discount factor, regularization, and episode budget). In my sweep, wider networks with lower learning rates did not outperform the 64-64 baseline. However, the longest training period with a moderate decaying epsilon produced the highest average validation returns among all logged runs. These experiments further emphasize the extreme sensitivities of Deep Reinforcement Learning (DRL) methods to various optimization parameters regardless of how simple the multi-layer perceptron (MLP) may be.

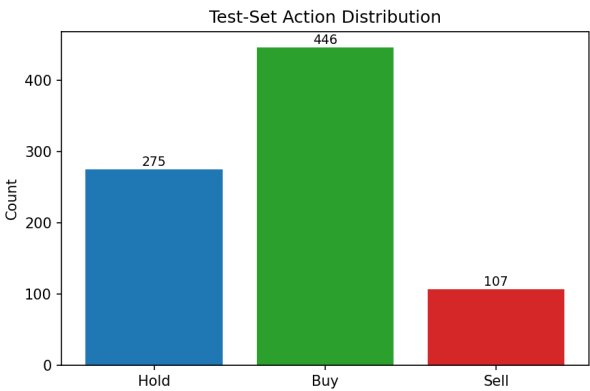


Fig. 4. Fig. 3. Distribution of greedy actions selected by the DQN agent on the test set.

Ex p.	Key settings	Train ret.	Test ret.	Test Sharpe	Test Max DD
-------	--------------	------------	-----------	-------------	-------------

1	64-64, lr=1e-3, 35 ep.	- 57.31 %	21.64 %	0.694	- 11.32 %
2	128-64, lr=5e-4, 35 ep.	- 5.61 %	5.47 %	0.224	- 14.19 %
3	gamma=0.995, slow eps	- 40.51 %	15.94 %	0.546	- 8.03%
4	lr=1e-4, L2=1e-3	- 63.85 %	10.61 %	0.370	- 7.68%
5	64-64, 50 episodes	- 57.20 %	32.88 %	1.017	- 7.18%
6*	Double-DQN, best val.	62.62 %	31.77 %	0.892	- 9.20%

Fig. 5. TABLE II. Hyperparameter sweep summary. Experiment 6 is the final Double-DQN configuration evaluated with the best validation weights (*).

This section provides some detail around the failures identified in Table II. Low learning rates (Experiment 4) will decrease the test drawdown over time but they will be very slow at improving the policy. The network size increase (Experiment 2) did not have enough episodes to generalize well from the data indicating that the model was not optimized sufficiently rather than having too few representations. Experiment 5 achieved the best raw test returns across the experiments and Experiment 6 had a high training return after applying reward scaling and double-dqn targets and thus will be used for most of the plots in this paper. However, buy-and-hold remains the top absolute-return and sharpe strategy on this bullish test fold (sharpe = 1.354, max drawdown = -18.90%) and the value of the dqn is in showing learned state conditional risk reduction using a clear method.

E. Discussion
The first limitation relates to how the project has been implemented as it is a long-only environment that does not allow for either short exposure or position sizing to be specified beyond being all-in/all-out. Therefore, there is limited expressiveness compared to a continuous action portfolio optimizer [5]. The second limitation relates to the fact that the use of daily bars omits any microstructure impacts that are likely to exist

when trading live. The third limitation relates to how rewards have been defined. The rewards have been based on immediately realized returns from changes to the portfolio's percent value and do not provide explicit penalties for draw-down nor reward objectives such as Risk Parity. Reward designs based on Sharpe Ratio or Differential Sharpe Ratios [6] are reasonable extensions of this work. Finally, due to the chronological nature of data splitting to prevent overfitting, the volatility and drift of the train and test sets may differ. However, using regime conditional evaluations would further strengthen any claims made about the ability of the model to learn and perform well across different regimes. Overall, I was able to meet my initial pedagogical goal by implementing a working DQN stack including Xavier/He-style initialization, back-propagation through time, replay buffer, double-dqn target updates and a double-dqn target update schedule and evaluating them on a standard-length market dataset. From an engineering standpoint, my command-line interface (CLI) `'python main.py --mode train'` logs every hyper-parameter combination and every metric generated by each evaluation into both csv format and human readable text format.

CONCLUSION AND FUTURE WORK

I built a from-scratch Deep Q-Network as an S&P 500 algorithmic trading strategy. This agent developed a meaningful hold/buy/sell strategy based on portfolio and technical characteristics that had a decreasing TD loss and increasing episode returns over time. On chronological, unseen data, this agent realized a 31.77 percent return with a Sharpe of .892 and max DD of -9.20%. The agent also outperformed buy-and-hold by having a lower draw-down (-18.90%) compared to its own return (buy-and-hold had a 35.19 percent return). For future work, I will investigate priority replay; dueling architectures; continuous position sizing through actor-critic methods; multi-asset portfolios; and risk-aware reward shaping.

REFERENCES

- [1] [1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [2] [2] V. Mnih et al., "Playing Atari with deep reinforcement learning," arXiv:1312.5602, 2013.
- [3] [3] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.
- [4] [4] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI*, 2016.
- [5] [5] Z. Jiang, D. Xu, and J. Liang, "A deep reinforcement learning framework for the financial portfolio management problem," arXiv:1706.10059, 2017.
- [6] [6] J. Moody and M. Saffell, "Learning to trade via direct reinforcement," *IEEE Trans. Neural Netw.*, vol. 12, no. 4, pp. 875–889, 2001.
- [7] [7] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE ICCV*, 2015.
- [8] [8] L.-J. Lin, "Self-improving reactive agents based on reinforcement learning, planning and teaching," *Machine Learning*, vol. 8, pp. 293–321, 1992.